Comparación empírica de Shell sort y Selección directa: tiempo, comparaciones e intercambios según el patrón de entrada

Empirical comparison of Shell sort and Selection sort: time, comparisons and swaps by input pattern

Joshep Alonso Condori Vera
Universidad Nacional Jorge Basadre Grohmann (UNJBG), Tacna, Perú
Programación Avanzada – Unidad 1
jacondoriv@unjbg.edu.pe
Octubre de 2026

Resumen. Se compararon dos algoritmos de ordenación implementados en C++: Selección directa y Shell sort con incrementos decrecientes hasta uno. Se analizaron cinco patrones de entrada (aleatorio, ascendente, descendente, casi ordenado y con duplicados), tres tamaños ($n = 1\,000$, $10\,000$ y $100\,000$) y 30 repeticiones por condición, para un total de 900 mediciones. Se registraron tiempo de ejecución, comparaciones y operaciones de intercambio o desplazamiento. Selección realizó exactamente $n(n-1)/2$ comparaciones en todos los patrones. Con $n = 100\,000$, la mediana del tiempo de Selección osciló entre 2 488.5 y 3 045.0 ms, mientras que Shell registró entre 3.8927 y 18.4522 ms. La aceleración de Shell frente a Selección varió de aproximadamente 154 a 752 veces. Los resultados evidenciaron un crecimiento temporal cercano al cuadrático para Selección y una marcada dependencia del patrón para Shell. Se documentó el procedimiento de medición y su reproducibilidad mediante semillas y un identificador de versión del código.
Palabras clave: algoritmos de ordenación; Shell sort; Selección directa; complejidad algorítmica; evaluación empírica; reproducibilidad.

Abstract. Two C++ sorting algorithms, Selection sort and Shell sort, were compared using five input patterns (random, ascending, descending, nearly sorted and duplicates), three input sizes ($n = 1,000$, $10,000$ and $100,000$), and 30 repetitions per condition, yielding 900 measurements. Execution time, comparisons and swaps or shifts were recorded. Selection sort performed exactly $n(n-1)/2$ comparisons regardless of input pattern. At $n = 100,000$, its median runtime ranged from 2,488.5 to 3,045.0 ms, while Shell sort ranged from 3.8927 to 18.4522 ms. Shell sort achieved approximately 154–752 times the median speed of Selection sort. The results showed near-quadratic time growth for Selection sort and substantial input-pattern sensitivity for Shell sort. Seeds and a code revision identifier were retained to support reproducibility.
Keywords: sorting algorithms; Shell sort; Selection sort; algorithmic complexity; empirical evaluation; reproducibility.

1. Introducción

La complejidad asintótica describe cómo aumenta el costo de un algoritmo conforme crece el tamaño de entrada, pero no determina por sí sola su tiempo de ejecución en una computadora específica. El patrón de los datos, las operaciones internas y el entorno de medición también pueden influir en el rendimiento observado.

Este estudio comparó Selección directa y Shell sort, dos algoritmos internos de ordenación, mediante un protocolo experimental reproducible. Se plantearon tres hipótesis des-

criptivas: (H1) Selección realiza el mismo número de comparaciones independientemente del patrón; (H2) el trabajo de Shell depende del orden inicial; y (H3) Shell es más rápido que Selección, con una ventaja creciente conforme aumenta n.

2. Método

2.1. Algoritmos y complejidad teórica

Selección directa. En cada iteración busca el menor elemento del segmento restante y lo sitúa en su posición definitiva. Su número exacto de comparaciones es

$$C(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}. \tag{1}$$

Por ello presenta complejidad temporal $\Theta(n^2)$ en mejor, promedio y peor caso. Requiere memoria auxiliar $O(1)$, no utiliza recursión y no es estable en su implementación usual con intercambios.

Shell sort. Generaliza inserción directa mediante subsecuencias separadas por incrementos k decrecientes hasta $k = 1$. Se empleó la secuencia de divisiones sucesivas por dos descrita en el modelo experimental. Su mejor caso para esta secuencia es del orden $O(n \log n)$ y su peor caso es $O(n^2)$; el comportamiento intermedio depende de los datos. Usa memoria auxiliar $O(1)$, es iterativo y no es estable.

2.2. Patrones y tamaños

Se generaron entradas con `std::mt19937`: aleatorio (enteros uniformes entre 0 y 1 000 000), ascendente, descendente, casi ordenado (intercambios aleatorios sobre una secuencia ascendente) y con duplicados (15 valores distintos). Se evaluaron $n \in \{1\,000, 10\,000, 100\,000\}$.

Se realizaron 30 repeticiones por combinación de algoritmo, patrón y tamaño:

$$2 \times 5 \times 3 \times 30 = 900. \tag{2}$$

La repetición r utilizó la semilla $12345 + r$, compartida por ambos algoritmos para comparar las mismas instancias.

2.3. Entorno computacional

Se utilizó una computadora con procesador Intel Core i3-8100, cuatro núcleos y cuatro hilos, frecuencia de 3.6 GHz, 16 GB de RAM y Windows 11. El código se compiló con MinGW GCC 6.3.0 y las opciones `-O2 -std=c++14`. El análisis de datos se realizó con Python; se comunicaron las versiones 3.13.15 y 3.1.0, sin identificación inequívoca de todos los paquetes estadísticos utilizados.

2.4. Métricas y procedimiento

Se registraron tiempo en milisegundos, comparaciones entre elementos, intercambios reales en Selección y desplazamientos en Shell. La profundidad de recursión fue cero en ambos casos, por tratarse de implementaciones iterativas. Para cada grupo se calcularon media, mediana, desviación estándar, mínimo y máximo; también se obtuvieron promedios de comparaciones y movimientos.

Según la documentación del experimento, una primera corrida con `std::chrono::steady_clock` presentó valores nulos por resolución insuficiente en el entorno utilizado. Se sustituyó por `QueryPerformanceCounter` en Windows y se repitieron las pruebas. Los CSV entregados corresponden a la corrida utilizada en el

presente análisis.

3. Resultados

3.1. Comparaciones y movimientos

En Selección se verificaron 499 500, 49 995 000 y 4 999 950 000 comparaciones para los tres tamaños, respectivamente, sin depender del patrón de entrada. En Shell las comparaciones cambiaron según la distribución de los elementos.

Tabla 1: Comparaciones y movimientos promedio con $n = 100\,000$.

Patrón	Comparaciones Selección	Comparaciones Shell	Intercambios Selección	Desplazamientos Shell
Aleatorio	4 999 950 000	4 363 853	99 989	2 914 329
Ascendente	4 999 950 000	1 500 006	0	0
Descendente	4 999 950 000	2 244 585	50 000	844 560
Casi ordenado	4 999 950 000	3 148 286	2 500	1 651 861
Duplicados	4 999 950 000	1 951 986	93 330	495 191

Fuente: elaboración propia a partir de las mediciones experimentales.

En el patrón ascendente Shell no necesitó desplazamientos, mientras que en el patrón aleatorio alcanzó el mayor número de comparaciones y desplazamientos entre los cinco casos examinados. En Selección, el número de intercambios varió, aunque el de comparaciones permaneció fijo.

Tabla 2: Mediana del tiempo de ejecución (ms) y aceleración de Shell frente a Selección.

Patrón	n	Selección	Shell	Razón
Aleatorio	1 000	0.3365	0.0909	3.7
	10 000	28.6617	1.3443	21.3
	100 000	2840.3850	18.4522	153.9
Ascendente	1 000	0.2952	0.0209	14.1
	10 000	28.1161	0.3094	90.9
	100 000	2926.0450	3.8927	751.7
Descendente	1 000	0.2873	0.0289	9.9
	10 000	25.4183	0.4195	60.6
	100 000	2488.5000	5.5786	446.1
Casi ordenado	1 000	0.2952	0.0571	5.2
	10 000	28.3567	0.9764	29.0
	100 000	3044.9500	14.4845	210.2
Duplicados	1 000	0.3126	0.0589	5.3
	10 000	28.4207	0.7530	37.7
	100 000	2799.0800	8.4601	330.9

Fuente: elaboración propia a partir de las mediciones experimentales.

3.2. Tiempo y variabilidad

Con $n = 100000$, Selección presentó medianas entre 2 488.5 y 3 045.0 ms; Shell, entre 3.8927 y 18.4522 ms. Para el patrón aleatorio, la mediana de Shell creció de 0.0909 a 1.3443 y 18.4522 ms, mientras que Selección creció de 0.3365 a 28.6617 y 2 840.3850 ms. La razón entre medianas aumentó con el tamaño de entrada en los cinco patrones.

Los estadísticos completos (media, mediana, desviación estándar, mínimo y máximo) para las 30 condiciones se conservan en `tabla_tiempos.csv`. La mediana se utilizó como indicador principal de comparación por su menor sensibilidad a valores extremos.

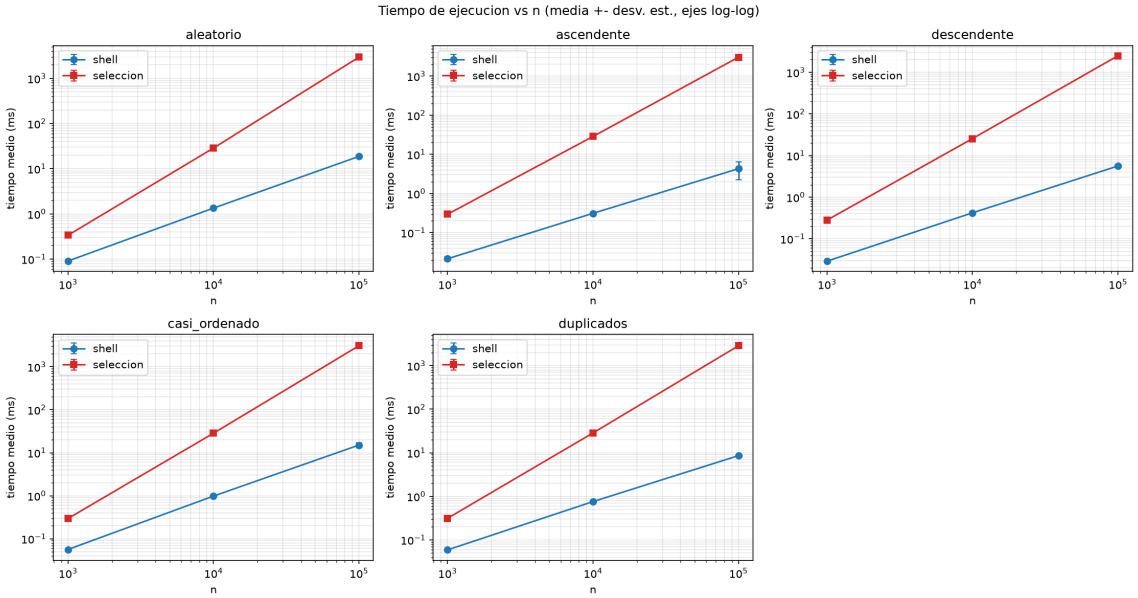


Figura 1: Tiempo de ejecución frente al tamaño de entrada: media y desviación estándar, en ejes logarítmicos, para los cinco patrones.

Fuente: elaboración propia a partir de las mediciones experimentales.

Las Figuras 1 y 2 presentan, respectivamente, las tendencias temporales y los conteos de comparaciones. Ambas figuras se elaboraron a partir de los resultados experimentales.

4. Discusión

4.1. Interpretación de las hipótesis

H1. Se confirmó para las comparaciones: Selección ejecutó $n(n-1)/2$ operaciones de comparación en todos los patrones. Sin embargo, sus tiempos no fueron idénticos; las diferencias pueden relacionarse con efectos del procesador, compilador y sistema operativo, no evaluados mediante perfilado.

H2. Se confirmó que Shell depende del patrón. Con $n = 100000$, realizó entre aproximadamente 1.5 y 4.4 millones de comparaciones. La entrada casi ordenada todavía requirió trabajo considerable, posiblemente por los intercambios entre posiciones distantes; esta explicación no fue verificada midiendo inversiones.

H3. Shell obtuvo menores medianas de tiempo en todas las condiciones. En el patrón

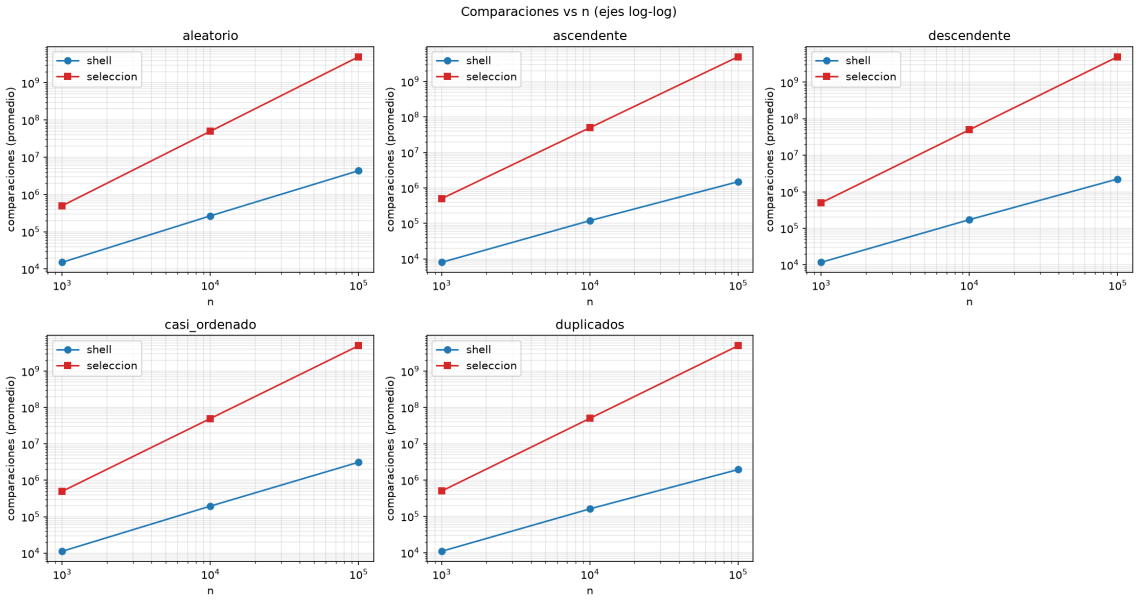


Figura 2: Número promedio de comparaciones frente al tamaño de entrada, en ejes logarítmicos, para los cinco patrones.

Fuente: elaboración propia a partir de las mediciones experimentales.

aleatorio la aceleración pasó de 3.7 a 21.3 y 153.9 veces conforme creció n. La tendencia empírica fue consistente con el crecimiento cuadrático de Selección, mientras que Shell mostró un crecimiento más moderado en el intervalo probado. Estos exponentes empíricos no constituyen demostraciones de cotas asintóticas.

El número de comparaciones por sí solo no predice el tiempo absoluto: los patrones de acceso a memoria, los movimientos y el costo de control de los bucles también pueden afectar el desempeño.

4.2. Limitaciones y amenazas a la validez

Las mediciones se realizaron en un solo equipo, con un compilador y una configuración de optimización. Solo se probaron tres tamaños y una secuencia de incrementos para Shell. Ninguno de los cinco patrones constituye necesariamente su peor caso teórico. La memoria auxiliar no fue medida directamente; su orden constante proviene del análisis de los algoritmos. Los valores atípicos de tiempo no fueron investigados mediante herramientas de perfilado.

5. Conclusiones

1. Selección directa realizó exactamente $n(n-1)/2$ comparaciones independientemente del patrón, en concordancia con su complejidad cuadrática.

2. Shell sort fue más rápido que Selección en todas las condiciones medidas y su ventaja aumentó con el tamaño de entrada.

3. El rendimiento de Shell dependió de la disposición inicial de los datos; el patrón ascendente fue el más favorable entre los evaluados con $n = 100\,000$.

4. Las comparaciones permiten describir el crecimiento del trabajo, pero no explican por sí solas todas las diferencias

de tiempo observadas.

5. La resolución del temporizador, las semillas y el registro de la versión del código resultaron relevantes para obtener mediciones reproducibles.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2nd ed.). Addison-Wesley.

Microsoft. (s. f.). *Acquiring high-resolution time stamps*. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/sysinfo/acquiring-high-resolution-time-stamps

Shell, D. L. (1959). A high-speed sorting procedure. *Communications of the ACM, 2*(7), 30–32. https://doi.org/10.1145/368370.368387

Anexo: reproducibilidad

Repositorio público: Consultar repositorio en GitHub.

Commit asociado a las mediciones: `fd75665`.

Semilla base: 12345. Para cada repetición r, se utilizó la semilla $12345 + r$ para generar datos pseudoaleatorios reproducibles.

Archivos de datos: `resultados.csv` y `tabla_tiempos.csv`.